AI-Powered Customer Support Automation System

Documentation Report

Course: Agentic AI
Assignment: DA-2 - Multi-Agent Customer Support System
GitHub Repository: https://github.com/bnssaanirudh/-AI-Powered-Customer-Support-Automation-System

Table of Contents

1. Project Overview

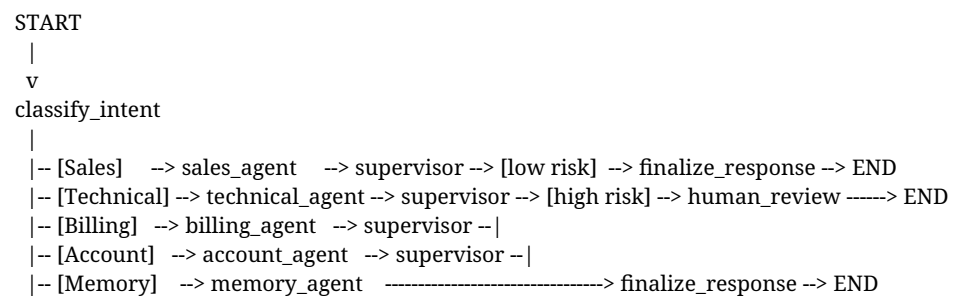
This project implements an AI-Powered Customer Support Automation System using LangGraph, LangChain, and the Groq LLM API. The system automates the end-to-end handling of customer support queries through a multi-agent architecture.

Core Capabilities

Capability	Description
Intent Classification	Automatically categorises incoming customer queries into departments
Conditional Routing	Directs each query to the correct specialist agent based on intent
RAG Retrieval	Retrieves grounded, fact-based context from a company knowledge base
Multi-Agent Response	Four specialist agents (Sales, Technical, Billing, Account) generate responses
Supervisor Validation	A supervisor node reviews and improves every response for quality and risk
Human-in-the-Loop	High-risk requests (refunds, cancellations) are paused for human approval
SQLite Memory	All conversation turns are persisted in a local SQLite database via LangGraph checkpointing
Memory Recall	A dedicated memory agent answers follow-up queries from conversation history

2. System Architecture

The system follows a directed acyclic graph (DAG) pattern orchestrated by LangGraph's StateGraph. Each node represents a discrete processing step and the graph uses conditional edges to implement dynamic routing.



Key Design Decisions

All agents route to the Supervisor - ensuring universal quality control and risk assessment for every query regardless of department.

Single HITL gate after Supervisor - the human_review node is triggered by an is_high_risk=True flag, covering refund, cancellation, account closure, compensation, and escalation.

Memory agent bypasses Supervisor - memory recall queries route directly to finalize_response since they answer historical facts rather than making new policy decisions.

LangGraph SQLite Checkpointer - SqliteSaver ensures the entire conversation state is automatically persisted and restored between queries.

3. Technology Stack

Component	Technology
Workflow Orchestration	LangGraph >= 0.1.0
LLM Framework	LangChain >= 0.2.0
Large Language Model	Groq llama-3.1-8b-instant via API
Embeddings	HuggingFace all-MiniLM-L6-v2 via sentence-transformers
Vector Store	ChromaDB (persistent, saved to ./chroma_db)
Memory / Checkpointing	SQLite via langgraph-checkpoint-sqlite
Text Splitting	LangChain RecursiveCharacterTextSplitter
Environment Config	python-dotenv

4. LangGraph Workflow Design

The workflow is defined in graph.py using StateGraph(CustomerSupportState).

Nodes Registered

Node Name	Function	Assignment Task
classify_intent	classify_intent_node()	Task 3 - Intent Classification
sales_agent	sales_agent_node()	Task 5 - Department Agent
technical_agent	technical_agent_node()	Task 5 - Department Agent
billing_agent	billing_agent_node()	Task 5 - Department Agent
account_agent	account_agent_node()	Task 5 - Department Agent

Node Name	Function	Assignment Task
memory_agent	memory_agent_node()	Task 7 - Memory Recall
supervisor	supervisor_node()	Task 9 - Supervisor
human_review	human_review_node()	Task 8 - HITL
finalize_response	finalize_response_node()	Task 6 - Output

Conditional Edge Functions

Router Function	Triggered After	Routes To
route_intent()	classify_intent	One of five agent nodes
hitl_check()	supervisor	human_review or finalize_response

Graph Compilation

The graph is compiled with interrupt_before=[“human_review”], which causes LangGraph to automatically pause execution and save the full state to SQLite before the human_review node runs.

5. Module Descriptions

state.py - Shared State Schema

Defines the CustomerSupportState TypedDict flowing through every node:

messages: Annotated[List[BaseMessage], operator.add] - Persistent conversation history using the add reducer

customer_query: str - The current customer query string

department: str - Classified intent: Sales, Technical, Billing, Account, or Memory

retrieved_context: str - RAG-retrieved document text injected into the agent prompt

proposed_response: str - Agent or Supervisor draft response

is_high_risk: bool - Risk flag set by the Supervisor node

human_approved: bool - The human operator’s decision (y/n)

final_response: str - Final delivered response sent to the customer

The messages field uses operator.add as its reducer, enabling LangGraph to append new messages to history without replacing the entire list. This is critical for memory recall to function correctly across sessions.

agents.py - Agent Implementations

classify_intent_node(): Uses keyword matching for memory queries; otherwise invokes the Groq LLM to classify the query as Sales, Technical, Billing, or Account.

department_agent(): Shared RAG-grounded response generator. Retrieves context via rag_pipeline.retrieve_context() and constructs a strict, context-only prompt to prevent hallucination.

sales_agent_node(), technical_agent_node(), billing_agent_node(), account_agent_node(): Thin wrappers calling department_agent() with the respective department name.

memory_agent_node(): Extracts conversation history from state[‘messages’], builds a plain-text history string, and prompts the LLM to answer the recall query. Explicitly clears retrieved_context to prevent stale RAG output from appearing in the terminal.

supervisor_node(): Reviews the proposed response, improves its tone and completeness, and determines if the request is HIGH RISK. Returns structured output with HIGH_RISK: YES/NO and IMPROVED_RESPONSE.

rag_pipeline.py - Retrieval-Augmented Generation

Two-phase RAG pipeline:

Ingestion: Loads all .txt files from documents/, splits into 500-character chunks with 50-character overlap using RecursiveCharacterTextSplitter, and embeds using all-MiniLM-L6-v2.

Retrieval: Fetches the top-2 most semantically similar chunks from ChromaDB and injects the retrieved text directly into the agent system prompt.

The retriever is lazily initialised as a module-level singleton to avoid re-building the vector index on every query.

graph.py - LangGraph Graph Builder

Assembles and compiles the full state machine. Key functions:

route_intent(): Maps state['department'] to the correct agent node name.

hitl_check(): Returns 'human_review' if is_high_risk is True, else 'finalize_response'.

human_review_node(): Reads state['human_approved'] and produces the final approved or rejected response string.

finalize_response_node(): Passes the proposed response directly to final_response.

main.py - Interactive Terminal Application

Loads environment variables and validates the Groq API key.

Creates the compiled workflow and generates workflow_diagram.png dynamically using app.get_graph(xray=True).draw_mermaid_png().

Accepts a Customer ID input to create a unique thread_id used as the SQLite checkpoint key.

Runs an interactive loop: sends queries through app.stream(), detects HITL breakpoints via state.next == ('human_review',), prompts for human approval, resumes via app.update_state() + app.stream(None, config), appends AIMessage to state for memory recall, and logs the SQLite save confirmation after every turn.

6. RAG Pipeline Implementation

Embedding Model

Uses all-MiniLM-L6-v2 from sentence-transformers via the langchain-huggingface integration. Produces 384-dimensional dense vector embeddings optimised for semantic similarity tasks.

Vector Store

ChromaDB is used as the persistent vector store, saved to ./chroma_db. This enables the vector index to persist across application restarts, so the embedding step only runs once.

Retrieval Strategy

search_type: cosine similarity (ChromaDB default)

k=2: retrieves the top 2 most relevant document chunks per query

Chunk size: 500 characters with 50-character overlap

Grounding Enforcement

The department agent system prompt:

"Answer the user query STRICTLY and ONLY using the facts provided in the Context below. DO NOT invent or guess any prices, policies, limits, or features that are not explicitly stated in the text. If the context does not contain the exact answer, you MUST state that you do not know but will escalate it. For policy or refund questions, always provide the complete details mentioned in the text."

Knowledge Base Files

File	Content
documents/pricing.txt	Subscription plans: Basic $29/mo, Pro $79/mo, Enterprise $199/mo; 20% annual discount
documents/policy.txt	Refund policy: 30-day window, prorated annual, human approval required; cancellations; escalations
documents/technical.txt	App crash causes: 50MB file limit, supported formats PDF/JPG/PNG; installation requirements
documents/faq.txt	Password reset, profile updates, invoice download

7. SQLite Memory System

Implementation

LangGraph's native SqliteSaver is used for checkpointing:

```
conn = sqlite3.connect('memory.db', check_same_thread=False)
memory = SqliteSaver(conn)
app = workflow.compile(checkpointer=memory, interrupt_before=['human_review'])
```

Database Tables

checkpoints - Complete snapshot of the graph state at each turn.

Column	Type	Description
thread_id	TEXT	Unique session ID e.g. customer_student_123
checkpoint_ns	TEXT	Namespace (defaults to empty string)
checkpoint_id	TEXT	Unique ID for this checkpoint
parent_checkpoint_id	TEXT	Link to previous checkpoint
type	TEXT	Serialisation format
checkpoint	BLOB	Serialised state snapshot
metadata	BLOB	Run metadata

writes - Individual node output writes for fine-grained state tracking.

Column	Type	Description
thread_id	TEXT	Session identifier
checkpoint_id	TEXT	Checkpoint this write belongs to
task_id	TEXT	Node task identifier
channel	TEXT	State field being written
blob	BLOB	Serialised field value

Memory Recall Mechanism

The memory_agent_node() reads state['messages'], which is populated because:

Each query adds a HumanMessage to the state before streaming.

After each response, main.py calls app.update_state() to append an AIMessage.

LangGraph checkpoints the state after every node, so the full message history is automatically recovered from SQLite on the next query.

8. Human-in-the-Loop (HITL) Workflow

Trigger Mechanism

Stage 1 - Supervisor Detection: The supervisor_node() analyses the user query and proposed response. If it detects keywords associated with high-risk operations (refund, cancellation, account closure, compensation, escalation), it sets is_high_risk = True.

Stage 2 - Graph Interrupt: The hitl_check() conditional edge reads is_high_risk. If True, it routes to human_review. Because the graph was compiled with interrupt_before=[‘human_review’], LangGraph saves the full state to SQLite and pauses execution before that node runs.

Human Approval Flow

```
>>> HITL BREAKPOINT TRIGGERED <<<
Supervisor flagged this request as HIGH RISK.
Proposed Response: [shown to operator]

Approve this request? (y/n):
 y --> app.update_state(human_approved=True)
      app.stream(None, config)  [resumes from SQLite checkpoint]
      human_review_node returns: Supervisor Approved: <response>

 n --> app.update_state(human_approved=False)
      app.stream(None, config)
      human_review_node returns: Request rejected by supervisor
```

Universal Coverage

The supervisor evaluates ALL agent outputs (Sales, Technical, Billing, Account) before the HITL gate. This means any query - regardless of which department handled it - can trigger a human review if the supervisor deems it high risk. This is the correct Task 8 implementation.

9. Supervisor Agent

The supervisor validates and improves every department agent’s response. Its structured output format is:

```
HIGH_RISK: YES
IMPROVED_RESPONSE: Thank you for reaching out. Your refund request for the annual
subscription has been received. Per our policy, annual subscription refunds are prorated.
As this is a high-value request, it requires supervisor approval before processing.
A member of our team will be in touch shortly.
```

The supervisor:

Rewrites responses for professional tone and completeness

Identifies high-risk categories: Refund, Cancellation, Account Closure, Compensation, Escalation

Ensures policy details (timeframes, approval requirements) are included in the response

Sets the is_high_risk flag in the state for the HITL routing decision

10. Task Compliance Matrix

Task	Requirement	Implementation	Status
Task 1	LangGraph multi-agent system	StateGraph with 9 nodes and conditional edges	Complete
Task 2	Customer support domain setup	Mock knowledge base in documents/ directory	Complete

Task	Requirement	Implementation	Status
Task 3	Intent classification	classify_intent_node() using LLM + keyword matching	Complete
Task 4	Conditional routing	route_intent() conditional edge function	Complete
Task 5	Four department agents	Sales, Technical, Billing, Account nodes	Complete
Task 6	RAG integration	ChromaDB + all-MiniLM-L6-v2 + strict grounding prompt	Complete
Task 7	SQLite memory	SqliteSaver + operator.add message reducer	Complete
Task 8	Human-in-the-loop	interrupt_before=['human_review'] + y/n approval	Complete
Task 9	Supervisor validation	supervisor_node() with structured HIGH_RISK output	Complete
Task 10	End-to-end demonstration	5 sample queries covering all agent paths	Complete

11. Setup and Installation

Prerequisites

Python 3.9 or higher

A valid Groq API key (free tier available at console.groq.com)

Installation Steps

```
# 1. Clone the repository
git clone https://github.com/bnssaanirudh/-AI-Powered-Customer-Support-Automation-System.git
cd AI-Powered-Customer-Support-Automation-System

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure environment - create a .env file in the project root
# Windows:
echo GROQ_API_KEY=your_key_here > .env
# Mac/Linux:
export GROQ_API_KEY=your_key_here

# 4. Run the application
python main.py
```

Note on memory.db

memory.db is created automatically on first run by LangGraph's SqliteSaver. You do not need to create it manually. schema.sql is included in the repository to document the database structure.

12. Sample Execution Walkthrough

Query 1 - Pricing (Sales Agent + RAG)

```
Enter Customer ID: student_123

[1] You: What are the pricing plans available for your software?

RETRIEVED CONTEXT:
```

ABC Technologies offers three subscription plans:
1. Basic Plan: $29/month - core features and email support
2. Pro Plan: $79/month - advanced analytics and 5 user accounts
3. Enterprise Plan: $199/month - dedicated manager and unlimited users
We offer a 20% discount on all plans if billed annually.

ROUTED TO: Sales
FINAL RESPONSE: ABC Technologies offers three subscription plans:
- Basic Plan at $29/month with core features and email support
- Pro Plan at $79/month with advanced analytics and 5 user accounts
- Enterprise Plan at $199/month with dedicated support and unlimited users
Annual billing provides a 20% discount on all plans.
[System] Conversation state successfully saved to SQLite memory.db checkpoint.

Query 2 - Password Reset (Account Agent + RAG)

[2] You: I forgot my account password.

ROUTED TO: Account
FINAL RESPONSE: To reset your password, please click the Forgot Password link on the login page. A reset link will be sent to your registered email address.
[System] Conversation state successfully saved to SQLite memory.db checkpoint.

Query 3 - Application Crash (Technical Agent + RAG)

[3] You: My application crashes whenever I upload a file.

RETRIEVED CONTEXT:
If the application crashes whenever you upload a file, it is usually due to a file size exceeding the 50MB limit or an unsupported file format. We support PDF, JPG, PNG.

ROUTED TO: Technical
FINAL RESPONSE: Application crashes during upload are typically caused by the file exceeding the 50MB size limit or an unsupported format. Please ensure your files are PDF, JPG, or PNG and are under 50MB. Compress or convert before uploading.
[System] Conversation state successfully saved to SQLite memory.db checkpoint.

Query 4 - Refund Request (Billing + HITL)

[4] You: I need a refund for my annual subscription.

>>> HITL BREAKPOINT TRIGGERED <<<
Supervisor flagged this request as HIGH RISK.
Proposed Response: Refunds are available within 30 days of purchase. For annual subscriptions, refunds are prorated. All refund requests require human approval.

Approve this request? (y/n): y
Resuming execution with human_approved = True...

ROUTED TO: Billing
FINAL RESPONSE: Supervisor Approved: Your refund request for the annual subscription has been received. Annual subscription refunds are prorated per our policy. Your request has been approved and will be processed shortly.
[System] Conversation state successfully saved to SQLite memory.db checkpoint.

Query 5 - Memory Recall

[5] You: What was my previous support issue?

ROUTED TO: Memory
FINAL RESPONSE: Based on our conversation history, your previous support issue was a refund request for your annual subscription.
[System] Conversation state successfully saved to SQLite memory.db checkpoint.

13. Knowledge Base Documents

documents/pricing.txt

Basic Plan: $29/month - core business management features and standard email support

Pro Plan: $79/month - advanced analytics, priority support, and up to 5 user accounts

Enterprise Plan: $199/month or custom - dedicated account manager, unlimited users, 24/7 phone support

Annual billing: 20% discount on all plans

documents/policy.txt

Refunds: Available within 30 days of purchase. All refund requests require human approval. Annual subscription refunds are prorated.

Cancellations: Can be requested at any time. Take effect at end of current billing cycle. Account closure requires human supervisor approval.

Escalations: Any compensation request or escalation to management must be reviewed by a human supervisor.

documents/technical.txt

Application Crashes: Usually caused by file size exceeding 50MB or unsupported file format. Supported formats: PDF, JPG, PNG.

Installation: Requires Windows 10 or macOS 11+, minimum 8GB RAM.

Configuration: Navigate to Settings tab; ensure API keys are correctly entered.

documents/faq.txt

Password Reset: Click Forgot Password on login page; reset link sent to registered email.

Profile Updates: Settings > Profile > Edit Profile.

Invoice Requests: Download from Billing section or contact Billing department for custom invoices.

14. Database Schema

The memory.db SQLite file is created automatically on first run by LangGraph’s SqliteSaver:

```
CREATE TABLE IF NOT EXISTS checkpoints (
    thread_id TEXT NOT NULL,
    checkpoint_ns TEXT NOT NULL DEFAULT '',
    checkpoint_id TEXT NOT NULL,
    parent_checkpoint_id TEXT,
    type TEXT,
    checkpoint BLOB,
    metadata BLOB,
    PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id)
);

CREATE TABLE IF NOT EXISTS writes (
    thread_id TEXT NOT NULL,
    checkpoint_ns TEXT NOT NULL DEFAULT '',
    checkpoint_id TEXT NOT NULL,
    task_id TEXT NOT NULL,
    idx INTEGER NOT NULL,
    channel TEXT NOT NULL,
    type TEXT,
    blob BLOB,
    PRIMARY KEY (thread_id, checkpoint_ns, checkpoint_id, task_id, idx)
);
```

15. Conclusion

This project successfully demonstrates all ten assignment tasks using a production-grade multi-agent architecture built with LangGraph.

Key strengths of the implementation:

True RAG Grounding - The strict system prompt prevents hallucination and forces the LLM to answer only from retrieved document context. Prices, policies, and technical specs are always retrieved from the knowledge base, never invented.

Universal Risk Gate - The supervisor evaluates every agent response, so no high-risk request can bypass human review regardless of which department handled it. This correctly implements Task 8 across all agent types.

Native LangGraph Memory - Using SqliteSaver and operator.add message reducers means conversation history is automatically maintained without any manual database queries. The state is always recovered from the last checkpoint.

Clean HITL Implementation - The interrupt_before mechanism is the correct LangGraph-native approach, ensuring state is safely persisted to SQLite before pausing for human input and seamlessly resumed after the operator decision.

Modular Architecture - Each concern (RAG, memory, routing, agents, supervisor) is cleanly separated into its own module, making the codebase readable, testable, and extensible.

GitHub Repository: https://github.com/bnssaanirudh/-AI-Powered-Customer-Support-Automation-System